ҚАЗАҚСТАН РЕСПУБЛИКАСЫНЫҢ ҒЫЛЫМ ЖӘНЕ ЖОҒАРЫ БІЛІМ МИНИСТРЛІГІ

Ш. ЕСЕНОВ АТЫНДАҒЫ ҚАСПИЙ ТЕХНОЛОГИЯЛАР ЖӘНЕ ИНЖИНИРИНГ УНИВЕРСИТЕТІ

«Педагогика» факультеті

6В01509 – Информатика (IP)

білім беру бағдарламасының

«Мобильді құрылғыларға арналған бағдарламалау» пәні бойынша

ТӘЖІРИБЕЛІК ЖҰМЫСТАРДЫ ОРЫНДАУҒА АРНАЛҒАН ӘДІСТЕМЕЛІК НҰСҚАУЛЫҚ

АҚТАУ 2026

ӘОЖ 004.42:371.388(072)

«Мобильді құрылғыларға арналған бағдарламалау» пәні бойынша тәжірибелік жұмыстарды орындауға арналған әдістемелік нұсқаулық

Университеттің академиялық кеңесімен **бекітілген**

«Педагогика» факультетінің академиялық бюросы **ұсынған** (20.11.24 № 5 хаттама)

Бюро төрағасы _____ Мамбетова А.И.

Құрастырған:

IF кафедрасының
қауымдастырылған
профессоры (доцент) _____ Шуакбаева Р.С.

Пікір беруші:

IF кафедрасының
профессоры _____ Абдыкеримова Э.А.

Аннотация: Бұл әдістемелік нұсқаулық «Мобильді құрылғыларға арналған бағдарламалау» пәні бойынша тәжірибелік жұмыстарды орындауға арналған. Нұсқаулықта Android/iOS платформаларына арналған мобильді қосымшаларды әзірлеудің негізгі қағидалары, пайдаланушы интерфейсін құру, деректермен жұмыс істеу және қосымшаларды тестілеу тәсілдері қарастырылған. Тәжірибелік тапсырмалар студенттердің бағдарламалау дағдыларын дамытуға, заманауи мобильді технологияларды меңгеруге және практикалық жобалар орындауға мүмкіндік береді.

№1. Тәжірибелік сабақ.

Тақырыбы: Android және iOS платформаларымен танысу

Сабақтын мақсаты:

• Студенттерге Android және iOS платформаларының архитектурасы, экожүйесі, әзірлеу құралдары және жариялау процестері туралы базалық түсінік қалыптастыру

• Платформа таңдауға әсер ететін факторларды (аудитория, бюджет, уақыт, технология) талдай алу дағдысын дамыту

Әдістемелік нұсқаулар:

- Android және iOS платформалары туралы ақпаратпен танысыңыз
- Әр платформаның ерекшеліктерін кесте түрінде салыстырыңыз
- Қай платформа қандай қолданушыларға ыңғайлы екенін анықтаңыз

Теориялық бөлім

Android платформасы

- Ашық бастапқы кодты операциялық жүйе
- Linux ядросы негізінде жасалған
- Әртүрлі өндірушілердің құрылғыларында қолданылады
- Қосымшалар Google Play дүкені арқылы таратылады

iOS платформасы

- Жабық бастапқы кодты операциялық жүйе
- Тек Apple компаниясының құрылғыларында жұмыс істейді
- Қауіпсіздігі жоғары
- Қосымшалар App Store арқылы таратылады

Android пен iOS салыстыру

- Ашық және жабық жүйе
- Құрылғылар саны
- Қолданушы интерфейсі
- Қосымша әзірлеу ерекшеліктері

Тапсырмалар:

1. Android және iOS платформаларына сипаттама беріңіз
2. Екі платформаның ұқсастықтары мен айырмашылықтарын кесте түрінде көрсетіңіз
3. Мобильді қосымша әзірлеу үшін қай платформаны таңдар едіңіз? Неге?

Бақылау сұрақтары:

1. Android платформасының негізгі ерекшелігі қандай?
2. iOS платформасының артықшылығы неде?
3. Android пен iOS арасындағы негізгі айырмашылықтар қандай?
4. Қай платформа ашық бастапқы кодты?
5. Қосымшалар қай дүкендер арқылы таратылады?

Студенттің өз бетінше орындайтын тапсырмалар:

- Android және iOS платформалары туралы қосымша ақпарат іздеу
- Әр платформада қолданылатын құрылғылар тізімін жасау
- Мобильді платформалар тақырыбына шағын конспект жазу

№2. Тәжірибелік сабақ.

Тақырыбы: Android Studio және Xcode орнату, бастапқы баптау

Қарастырылатын сұрақтар:

1. Android Studio IDE құрылымы.
2. Xcode ортасының ерекшеліктері.
3. Алғашқы жобаны жасау кезеңдері.
4. Эмуляторды іске қосу.

Әдістемелік нұсқаулар:

1) **Android Studio:** Java/Kotlin тілдеріне негізделген, Android жүйесіне арналған IDE.
2) **Xcode:** Swift/Objective-C тілдерінде iOS қосымшаларын әзірлеу үшін қолданылады.
3) Алғашқы жоба «Hello World» қосымшасынан басталады.

Кесте 1 - Әзірлеу орталарын салыстыру:

Көрсеткіш	Android Studio	Xcode
Платформа	Android	iOS/macOS
Тілдер	Java, Kotlin	Swift, Objective-C
Артықшылықтары	Ашық, икемді, тегін	Apple экожүйесіне толық интеграцияланған
Кемшілігі	Кейде баяу жұмыс істейді	Тек MacOS-та қолжетімді

Тапсырмалар:

1. Android Studio-ны орнатып, алғашқы жоба жасаңыз.
2. Xcode ортасында жаңа жоба ашып, интерфейсті тексеріңіз.
3. «Hello World» мәтінін экранға шығарыңыз.

4. Эмулятор арқылы жобаны іске қосыңыз.

Бақылау сұрақтары:

1. Android Studio қай тілдерде қосымша жасайды?
2. Xcode қандай жүйеге арналған?
3. Эмулятордың қызметі қандай?
4. Алғашқы жобаны құрудың қадамдарын атаңыз.
5. Android Studio мен Xcode айырмашылығы неде?

Студенттің өз бетінше орындайтын тапсырмалар:

1. Өз құрылғыңызға эмулятор арқылы қосымша орнатып көріңіз.
2. «Hello World» қосымшасын өзгертіп, өз атыңызды шығарыңыз.
3. Android пен iOS-тағы жоба құрылымын салыстырыңыз.

№3. Тәжірибелік сабақ.

Тақырыбы: Қарапайым мобильді қосымша жасау (Hello World)

Сабақтың мақсаты

- Мобильді қосымша ұғымымен танысу
- Android Studio немесе Xcode ортасында алғашқы мобильді қосымшаны құру
- Hello World қосымшасы арқылы жобаның құрылымын түсіну
- Мобильді қосымшаның негізгі архитектуралық үлгілерін меңгеру

Қарастырылатын сұрақтар

- Мобильді қосымша деген не
- Android және iOS платформаларының ерекшеліктері
- Hello World қосымшасының мәні
- Жобаның негізгі файлдары мен олардың қызметі
- Мобильді қосымшалардағы архитектура ұғымы

Әдістемелік нұсқаулар

- Сабақты бастамас бұрын Android Studio (Android үшін) немесе Xcode (iOS үшін) орнатылғанына көз жеткізу
- Жаңа жоба құру кезінде стандартты шаблонды таңдау
- Бағдарламалау тілі ретінде Android үшін Kotlin (немесе Java), iOS үшін Swift қолдану
- Экранға қарапайым мәтін шығару арқылы қосымшаның жұмысын тексеру
- Қосымшаны эмуляторда немесе нақты құрылғыда іске қосу

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Қосымшаны модель, көрініс және басқару логикасына бөлу арқылы құрылымдау
MVP (Model–View–Presenter)	View пен Model арасындағы байланысты Presenter арқылы ұйымдастыру
MVVM (Model–View–ViewModel)	View мен логиканы ViewModel арқылы байланыстыратын заманауи үлгі
Clean Architecture	Қосымшаны қабаттарға бөліп, тәуелділіктерді азайтуға бағытталған үлгі

Тапсырмалар

- Android Studio немесе Xcode ортасында жаңа жоба құру
- Hello World мәтінін экранға шығаратын қосымша жасау
- Қосымшаның негізгі файлдарын (MainActivity / ViewController) анықтау
- Қолданылған архитектуралық үлгіні сипаттау
- Қосымшаны эмуляторда іске қосып, скриншот жасау

Бақылау сұрақтары

- Hello World қосымшасының негізгі мақсаты қандай
- Мобильді қосымшаның негізгі компоненттері қандай
- Android пен iOS жобаларының айырмашылығы неде
- MVC және MVVM архитектураларының айырмашылығы қандай
- Архитектуралық үлгілерді қолданудың артықшылығы неде

Студенттің өздік жұмыстары

- Hello World қосымшасын өз бетінше жетілдіру (түстер, қаріп, орналасу)
- Қосымшаға бір батырма қосып, оның әрекетін сипаттау
- MVC немесе MVVM архитектурасы бойынша қысқаша конспект дайындау
- Мобильді қосымшаларда архитектураны дұрыс таңдаудың маңыздылығы туралы шағын эссе жазу

№4 .Тәжірибелік сабақ.

Тақырыбы: Пайдаланушы интерфейсін (UI) жобалау: түймелер, мәтін өрістері және бейнелер

Сабақтың мақсаты

- Пайдаланушы интерфейсі (UI) ұғымын түсіндіру

- Мобильді қосымшаларда негізгі UI элементтерін қолдануды үйрету
- Түймелер, мәтін өрістері және бейнелермен жұмыс істеу дағдыларын қалыптастыру
- Қарапайым интерфейсті жобалау принциптерін меңгеру

Қарастырылатын сұрақтар

- Пайдаланушы интерфейсі (UI) деген не
- UI және UX ұғымдарының айырмашылығы
- Мобильді қосымшалардағы негізгі интерфейс элементтері
- Түймелердің (Button) қызметі мен түрлері
- Мәтін өрістерінің (TextField / EditText) қолданылуы
- Бейнелермен (ImageView) жұмыс істеу ерекшеліктері
- Интерфейс орналасуын басқару (Layout)

Әдістемелік нұсқаулар

- Жұмысты Android Studio немесе Xcode ортасында орындау
- Жобаға графикалық интерфейс элементтерін визуалды редактор немесе код арқылы қосу
- Интерфейс элементтерін логикалық және ыңғайлы орналастыру
- Экран өлшемдеріне бейімделетін интерфейс құруға назар аудару
- Интерфейстің түс үйлесімділігі мен оқылымдылығын сақтау
- Қосымшаны эмуляторда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Интерфейсті (View) логикадан және деректерден бөлуге арналған үлгі
MVP (Model–View–Presenter)	UI логикасын Presenter арқылы басқару
MVVM (Model–View–ViewModel)	Интерфейс пен деректер арасындағы байланысты ViewModel қамтамасыз етеді
Clean Architecture	Қосымшаның тұрақты, кеңейтілетін құрылымын қалыптастыруға бағытталған

Тапсырмалар

- Жобаға бір түйме (Button) қосып, оған мәтін орнату
- Мәтін енгізуге арналған өріс (TextField / EditText) қосу
- Экранға сурет (ImageView) орналастыру

- Интерфейс элементтерін дұрыс орналасу контейнері арқылы реттеу
- Түймені басу кезінде мәтінді өзгерту әрекетін іске асыру
- Қосымшаны эмуляторда іске қосып, нәтижесін тексеру

Бақылау сұрақтары

- Пайдаланушы интерфейсі дегеніміз не
- UI мен UX айырмашылығы неде
- Түймелер қандай қызмет атқарады
- Мәтін өрістерінің негізгі түрлері қандай
- Бейнелерді қосымшада қолданудың маңызы неде
- Интерфейсті жобалау кезінде қандай талаптар сақталуы керек

Студенттің өздік жұмыстары

- Интерфейсті өз бетінше жетілдіру (түстер, қаріптер, өлшемдер)
- Қосымшаға екі немесе одан да көп түйме қосу
- Пайдаланушы енгізген мәтінді экранда көрсету
- UI дизайнының маңыздылығы туралы қысқаша рефлексия жазу
- Жақсы және нашар интерфейске мысалдар келтіру

№5 .Тәжірибелік сабақ.

Тәжірибелік сабақ

Тақырыбы: Белсенділіктер (Activity) және фрагменттер (Fragment) арасындағы өзара әрекеттесу

Сабақтың мақсаты

- Android қосымшасындағы Activity және Fragment ұғымдарын түсіндіру
- Activity мен Fragment арасындағы байланысты ұйымдастыру жолдарын меңгеру
- Экранды модульдерге бөліп жобалау дағдыларын қалыптастыру
- Қосымшаның икемділігі мен қайта қолданбалылығын арттыру

Қарастырылатын сұрақтар

- Activity деген не және оның қызметі
- Fragment ұғымы және оның артықшылықтары
- Activity мен Fragment айырмашылығы
- Fragment өмірлік циклі
- Activity ↔ Fragment арасындағы деректер алмасу
- FragmentManager және FragmentTransaction ұғымдары
- Бір Activity ішінде бірнеше Fragment қолдану

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Жобада кемінде бір Activity және бір Fragment құру
- Fragment-ті Activity ішіне контейнер арқылы орналастыру
- FragmentManager көмегімен Fragment қосу, ауыстыру немесе жою
- Fragment пен Activity арасында интерфейс немесе Bundle арқылы байланыс орнату
- Қосымшаны әртүрлі экран өлшемдерінде тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Activity мен Fragment View рөлін атқарып, логика Controller арқылы басқарылады
MVP (Model–View–Presenter)	Fragment интерфейс ретінде қолданылады, логика Presenter-де орналасады
MVVM (Model–View–ViewModel)	Fragment деректерді ViewModel арқылы алады, Activity басқарушы рөл атқарады
Clean Architecture	UI қабаты (Activity/Fragment) бизнес логикадан толық бөлінеді

Тапсырмалар

- Жаңа Android жобасын құру
- Бір Activity және бір Fragment жасау
- Fragment-ті Activity ішіне қосу
- Fragment ішіндегі түймені басу арқылы Activity-дегі мәтінді өзгерту
- Екі Fragment арасында ауысуды ұйымдастыру
- Қосымшаны эмуляторда іске қосып, нәтижесін тексеру

Бақылау сұрақтары

- Activity мен Fragment айырмашылығы неде
- Fragment қолданудың артықшылықтары қандай
- Fragment өмірлік циклі Activity циклінен қалай ерекшеленеді
- Activity мен Fragment арасында деректер қалай беріледі
- FragmentManager қандай қызмет атқарады
- Бір Activity ішінде бірнеше Fragment қолданудың пайдасы неде

Студенттің өздік жұмыстары

- Екі Fragment-тен тұратын интерфейс жобалау
- Fragment арқылы экранды ауыстыру логикасын жетілдіру

- Activity мен Fragment арасындағы байланыс тәсілдерін салыстыру
- Fragment қолданылатын танымал мобильді қосымшаларға мысал келтіру
- Activity мен Fragment тақырыбы бойынша қысқаша конспект дайындау

№6. Тәжірибелік сабақ.

Тәжірибелік сабақ

Тақырыбы: Мобильді қосымшада деректерді енгізу және шығару (EditText, TextView, Toast, Snackbar)

Сабақтың мақсаты

- Мобильді қосымшаларда пайдаланушыдан деректерді енгізу тәсілдерін меңгерту
- EditText және TextView элементтерімен жұмыс істеуді үйрету
- Toast және Snackbar арқылы хабарламалар көрсетуді тәжірибеде қолдану
- Пайдаланушымен интерактивті өзара әрекеттесу дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Мобильді қосымшаларда деректерді енгізу және шығару ұғымы
- EditText элементінің қызметі және қолданылуы
- TextView элементінің қызметі
- Toast хабарламасының ерекшеліктері
- Snackbar хабарламасының артықшылықтары
- Toast пен Snackbar айырмашылығы
- Пайдаланушы әрекетіне жауап беру принциптері

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Интерфейске кемінде бір EditText және бір TextView қосу
- Пайдаланушы енгізген мәтінді алу және өңдеу
- Түйме басылған кезде енгізілген деректерді TextView арқылы көрсету
- Toast және Snackbar көмегімен хабарлама шығару
- Интерфейстің ыңғайлылығы мен қолдануға жеңілдігін ескеру
- Қосымшаны эмуляторда немесе нақты құрылғыда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	EditText пен TextView View ретінде қолданылады, логика Controller-де орналасады

Архитектуралық үлгі	Сипаттамасы
MVP (Model–View–Presenter)	Пайдаланушы енгізген деректер Presenter арқылы өңделеді
MVVM (Model–View–ViewModel)	EditText-тен алынған деректер ViewModel арқылы View-ге беріледі
Clean Architecture	UI элементтері бизнес логикадан толық ажыратылады

Тапсырмалар

- Интерфейске EditText және TextView қосу
- Пайдаланушы енгізген мәтінді алу
- Алынған мәтінді TextView-де көрсету
- Түйме басылған кезде Toast хабарламасын шығару
- Snackbar көмегімен әрекетке жауап беретін хабарлама көрсету
- Toast пен Snackbar айырмашылығын тәжірибе арқылы көрсету

Бақылау сұрақтары

- EditText қандай мақсатта қолданылады
- TextView элементінің негізгі қызметі қандай
- Toast хабарламасы қашан қолданылады
- Snackbar хабарламасының Toast-тан айырмашылығы неде
- Пайдаланушымен интерактивті байланыс не үшін қажет
- Деректерді енгізу кезінде қандай қателіктерге жол бермеу керек

Студенттің өздік жұмыстары

- EditText арқылы бірнеше дерек енгізуге арналған форма жасау
- Пайдаланушы қате дерек енгізген жағдайда Toast немесе Snackbar көрсету
- Snackbar-ға әрекет батырмасын (Action) қосу
- Деректерді енгізу және шығару тақырыбы бойынша қысқаша конспект жазу
- Қолданушыға ыңғайлы интерфейс жасау бойынша ұсыныстар дайындау

№7. Тәжірибелік сабақ.

Тақырыбы: Intent және SharedPreferences көмегімен деректерді сақтау және жіберу

Сабақтың мақсаты

- Android қосымшасында деректерді уақытша және тұрақты сақтау тәсілдерін меңгерту
- Activity арасында деректерді Intent арқылы жіберуді үйрету
- SharedPreferences көмегімен қарапайым деректерді сақтау және оқу дағдыларын қалыптастыру
- Қосымшаның пайдаланушыға бейімделу мүмкіндігін арттыру

Қарастырылатын сұрақтар

- Intent ұғымы және оның түрлері
- Explicit және Implicit Intent айырмашылығы
- Activity арасында деректерді жіберу принципі
- SharedPreferences деген не
- SharedPreferences арқылы деректерді сақтау және оқу
- Intent пен SharedPreferences қолдану жағдайларын салыстыру
- Деректердің сақталу уақыты мен қауіпсіздігі

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Жобада кемінде екі Activity құру
- Бір Activity-ден екіншісіне Intent арқылы дерек жіберу
- EditText арқылы енгізілген деректерді Intent-ке қосу
- SharedPreferences көмегімен мәтіндік немесе логикалық деректерді сақтау
- Қосымшаны қайта ашқан кезде сақталған деректерді көрсету
- Қосымшаның жұмысын эмуляторда немесе нақты құрылғыда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Intent Controller арқылы View-лер арасындағы байланысты қамтамасыз етеді
MVP (Model–View–Presenter)	Presenter деректерді Intent және SharedPreferences арқылы өңдейді
MVVM (Model–View–ViewModel)	ViewModel деректерді сақтап, View-ге қайта ұсынады
Clean Architecture	Деректерді сақтау механизмі UI қабатынан бөлінеді

Тапсырмалар

- Екі Activity-ден тұратын Android қосымшасын құру

- Бірінші Activity-де EditText арқылы дерек енгізу
- Intent көмегімен деректі екінші Activity-ге жіберу
- Екінші Activity-де қабылданған деректі экранда көрсету
- SharedPreferences арқылы пайдаланушы аты немесе параметрін сақтау
- Қосымшаны қайта іске қосқанда сақталған деректі көрсету

Бақылау сұрақтары

- Intent дегеніміз не
- Explicit Intent пен Implicit Intent айырмашылығы қандай
- Activity арасында деректер қалай жіберіледі
- SharedPreferences қандай деректерді сақтауға арналған
- Intent пен SharedPreferences қай жағдайда қолданылады
- Деректерді сақтау кезінде қандай қауіпсіздік талаптары бар

Студенттің өздік жұмыстары

- Intent арқылы бірнеше деректі бір Activity-ден екіншісіне жіберу
- SharedPreferences көмегімен қолданушы параметрлерін сақтау
- Қосымшаның тілін немесе тақырыбын (theme) сақтайтын функция қосу
- Intent және SharedPreferences тақырыбы бойынша қысқаша конспект жазу
- Деректерді сақтау тәсілдерін салыстыратын кесте құру

№8. Тәжірибелік сабақ.

Тақырыбы: Мобильді қосымшалардағы мультимедиа (аудио, бейне, суреттермен жұмыс)

Сабақтың мақсаты

- Мобильді қосымшаларда мультимедиа ұғымын түсіндіру
- Аудио, бейне және суреттермен жұмыс істеу тәсілдерін меңгеру
- Мультимедиалық элементтерді қосымша интерфейсіне кірістіру дағдыларын қалыптастыру
- Қосымшаның көрнекілігі мен функционалдылығын арттыру

Қарастырылатын сұрақтар

- Мультимедиа деген не
- Мобильді қосымшаларда мультимедианы қолдану маңызы
- Суреттермен (ImageView) жұмыс істеу
- Аудио файлдарды ойнату (MediaPlayer)

- Бейне файлдарды көрсету (VideoView)
- Ресурстар папкасымен (res) жұмыс істеу
- Мультимедиа файлдарының форматтары
- Мультимедианы қолдану кезіндегі өнімділік мәселелері

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Жобаға сурет, аудио және бейне файлдарын ресурс ретінде қосу
- Суретті ImageView көмегімен экранда көрсету
- MediaPlayer арқылы аудио файлды ойнату
- VideoView көмегімен бейне файлды іске қосу
- Мультимедиалық элементтерді түймелер арқылы басқару
- Қосымшаны эмуляторда немесе нақты құрылғыда тексеру
- Файл өлшеміне және құрылғы ресурстарына назар аудару

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Мультимедиа элементтері View рөлін атқарады, басқару логикасы Controller-де орналасады
MVP (Model–View–Presenter)	Аудио және бейне ойнату логикасы Presenter арқылы басқарылады
MVVM (Model–View–ViewModel)	Мультимедиа күйі ViewModel арқылы бақыланады
Clean Architecture	Мультимедиа өңдеу логикасы UI қабатынан бөлінеді

Тапсырмалар

- Жобаға бір сурет файлын қосып, экранда көрсету
- Аудио файлды ойнататын түйме қосу
- Бейне файлды VideoView арқылы көрсету
- Мультимедианы іске қосу және тоқтату батырмаларын ұйымдастыру
- Мультимедиалық элементтерді дұрыс орналастыру
- Қосымшаның жұмысын эмуляторда тексеру

Бақылау сұрақтары

- Мультимедиа дегеніміз не
- Мобильді қосымшаларда мультимедианы қолданудың артықшылықтары қандай

- ImageView қандай қызмет атқарады
- MediaPlayer не үшін қолданылады
- VideoView көмегімен қандай мүмкіндіктер іске асырылады
- Мультимедиа файлдарымен жұмыс істеу кезінде қандай шектеулер бар

Студенттің өздік жұмыстары

- Бірнеше суреттен тұратын галерея жасау
- Аудио ойнатқыштың қарапайым интерфейсін құру
- Бейне ойнатуға басқару элементтерін қосу
- Мультимедианы қолданудың білім беру немесе ойын қосымшаларындағы мысалдарын зерттеу
- Мобильді қосымшалардағы мультимедиа тақырыбы бойынша қысқаша конспект жазу

№9. Тәжірибелік сабақ.

Тақырыбы: Тізімдермен жұмыс (RecyclerView, ListView)

Сабақтың мақсаты

- Мобильді қосымшаларда тізімдермен жұмыс істеу ұғымын түсіндіру
- ListView және RecyclerView компоненттерінің қызметін меңгеру
- Деректерді тізім түрінде көрсету дағдыларын қалыптастыру
- RecyclerView қолданудың артықшылықтарын тәжірибеде көрсету

Қарастырылатын сұрақтар

- Мобильді қосымшалардағы тізім ұғымы
- ListView компонентінің қызметі
- RecyclerView компонентінің ерекшеліктері
- Adapter ұғымы және оның рөлі
- ViewHolder түсінігі
- ListView пен RecyclerView айырмашылығы
- Үлкен көлемді деректермен жұмыс істеу принциптері

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Интерфейске ListView немесе RecyclerView қосу
- Қарапайым деректер тізімін (мәтіндер жиыны) дайындау
- Adapter құру және оны тізім компонентіне байланыстыру
- RecyclerView үшін LayoutManager орнату
- Тізім элементін басу оқиғасын (onClick) ұйымдастыру
- Қосымшаны эмуляторда немесе нақты құрылғыда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Тізім деректері Model-де, көрсету View-де, басқару Controller-де орналасады
MVP (Model–View–Presenter)	Presenter тізім деректерін өңдеп, View-ге ұсынады
MVVM (Model–View–ViewModel)	RecyclerView деректері ViewModel арқылы жаңартылып отырады
Clean Architecture	Тізім логикасы мен деректер UI қабатынан бөлек ұйымдастырылады

Тапсырмалар

- ListView көмегімен қарапайым мәтіндер тізімін көрсету
- RecyclerView пайдаланып тізім құру
- RecyclerView үшін Adapter және ViewHolder жасау
- Тізім элементін басқан кезде хабарлама (Toast) шығару
- ListView пен RecyclerView жұмысын салыстыру
- Қосымшаны эмуляторда іске қосып, нәтижесін тексеру

Бақылау сұрақтары

- ListView пен RecyclerView айырмашылығы неде
- Adapter қандай қызмет атқарады
- ViewHolder не үшін қажет
- RecyclerView не себепті тиімді болып саналады
- Үлкен тізімдермен жұмыс істеу кезінде қандай қиындықтар туындайды
- Тізім элементімен әрекеттесу қалай жүзеге асады

Студенттің өздік жұмыстары

- RecyclerView арқылы сурет пен мәтіннен тұратын тізім жасау
- Тізім элементтерін динамикалық түрде өзгерту
- RecyclerView-ге бірнеше түрлі элемент түрін (item type) қосу
- ListView және RecyclerView салыстыру кестесін құру
- Тізімдермен жұмыс тақырыбы бойынша қысқаша конспект дайындау

№10. Тәжірибелік сабақ.

Тақырыбы: Жергілікті деректер базасын қолдану (SQLite, Room)

Сабақтың мақсаты

- Мобильді қосымшаларда жергілікті деректер базасы ұғымын түсіндіру
- SQLite деректер базасымен жұмыс істеу негіздерін меңгеру
- Room кітапханасының артықшылықтарын көрсету
- Деректерді сақтау, оқу, жаңарту және жою (CRUD) операцияларын орындау дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Жергілікті деректер базасы деген не
- SQLite деректер базасының ерекшеліктері
- Room кітапханасының қызметі
- SQLite мен Room айырмашылығы
- Entity, DAO, Database ұғымдары
- CRUD операциялары
- Деректердің қауіпсіздігі мен тұтастығы

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Жобаға SQLite немесе Room тәуелділіктерін қосу
- Деректер базасының құрылымын (кестелер, өрістер) жобалау
- Entity класын анықтау
- DAO интерфейсі арқылы деректермен жұмыс істеу
- Room Database класын құру
- Деректерді қосымшада көрсету (RecyclerView немесе TextView арқылы)
- Қосымшаны эмуляторда немесе нақты құрылғыда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Деректер базасы Model рөлін атқарады, UI View арқылы көрсетіледі
MVP (Model–View–Presenter)	Presenter деректерді SQLite немесе Room арқылы өңдейді
MVVM (Model–View–ViewModel)	ViewModel деректер базасымен тікелей жұмыс істеп, View-ге дерек ұсынады
Clean Architecture	Деректер қабаты UI-ден толық бөлініп, тәуелсіз ұйымдастырылады

Тапсырмалар

- SQLite немесе Room негізінде деректер базасын құру
- Қарапайым кесте (мысалы, пайдаланушы немесе тапсырма) жасау
- Деректерді базаға қосу
- Сақталған деректерді экранда көрсету
- Деректерді жаңарту және жою операцияларын орындау
- SQLite мен Room қолдану ыңғайлылығын салыстыру

Бақылау сұрақтары

- Жергілікті деректер базасы не үшін қажет
- SQLite деректер базасының артықшылықтары қандай
- Room кітапханасы SQLite-ты қалай жеңілдетеді
- Entity және DAO ұғымдары нені білдіреді
- CRUD операциялары дегеніміз не
- Жергілікті деректерді сақтау кезінде қандай шектеулер бар

Студенттің өздік жұмыстары

- Room көмегімен шағын деректер базасын жобалау
- Бірнеше кестеден тұратын деректер құрылымын құру
- Деректерді RecyclerView арқылы көрсету
- SQLite және Room салыстырмалы талдауын жазу
- Жергілікті деректер базасының мобильді қосымшалардағы рөлі туралы қысқаша эссе дайындау

№11. Тәжірибелік сабақ.

Тақырыбы: Firebase арқылы бұлттық деректер базасымен жұмыс істеу

Сабақтың мақсаты

- Бұлттық деректер базасы ұғымын түсіндіру
- Firebase платформасының мүмкіндіктерімен танысу
- Firebase Realtime Database және Cloud Firestore арқылы деректермен жұмыс істеуді үйрену
- Мобильді қосымшаларда деректерді онлайн режимде сақтау және синхрондау дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Бұлттық деректер базасы деген не
- Firebase платформасының негізгі қызметтері
- Firebase Realtime Database ерекшеліктері

- Cloud Firestore айырмашылығы мен артықшылықтары
- Firebase жобасын құру және қосымшаға қосу
- Деректерді жазу, оқу, жаңарту және жою
- Онлайн және оффлайн режимде деректермен жұмыс

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Firebase Console арқылы жаңа жоба құру
- Қосымшаны Firebase жобасына байланыстыру
- Firebase тәуелділіктерін (dependencies) Gradle файлына қосу
- Realtime Database немесе Firestore деректер құрылымын жобалау
- Қосымша арқылы деректерді бұлтқа жазу және оқу
- Қосымшаны эмуляторда немесе нақты құрылғыда тестілеу
- Деректер қауіпсіздігі ережелеріне (Security Rules) назар аудару

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Firebase деректері Model ретінде қолданылады
MVP (Model–View–Presenter)	Presenter Firebase-тен алынған деректерді өңдейді
MVVM (Model–View–ViewModel)	ViewModel Firebase деректер ағынын басқарады
Clean Architecture	Бұлттық деректер қабаты UI-ден толық ажыратылады

Тапсырмалар

- Firebase Console арқылы жаңа жоба құру
- Android қосымшасын Firebase жобасына қосу
- Firebase Realtime Database немесе Firestore таңдау
- Қарапайым деректерді (мысалы, пайдаланушы аты) бұлтқа сақтау
- Бұлттан деректерді оқып, экранда көрсету
- Бір деректі жаңарту немесе жою операциясын орындау
- Firebase пен жергілікті деректер базасын салыстыру

Бақылау сұрақтары

- Бұлттық деректер базасының артықшылықтары қандай
- Firebase Realtime Database пен Firestore айырмашылығы неде

- Firebase қандай жағдайларда тиімді қолданылады
- Онлайн және оффлайн деректер синхрондауы қалай жүзеге асады
- Firebase Security Rules не үшін қажет
- Firebase қолдану кезінде қандай тәуекелдер бар

Студенттің өздік жұмыстары

- Firebase көмегімен шағын чат немесе жазбалар қосымшасын жасау
- Бірнеше пайдаланушыға ортақ деректер базасын ұйымдастыру
- Firebase деректер құрылымын диаграмма түрінде көрсету
- Firebase және SQLite/Room салыстырмалы талдауын жазу
- Бұлттық деректер базасының мобильді қосымшалардағы рөлі туралы қысқаша эссе дайындау

№12. Тәжірибелік сабақ.

Тақырыбы: Геолокация және карталар (Google Maps API, GPS деректерін пайдалану)

Сабақтың мақсаты

- Геолокация ұғымын және оның мобильді қосымшалардағы рөлін түсіндіру
- GPS деректерін пайдалану принциптерін меңгеру
- Google Maps API көмегімен картамен жұмыс істеуді үйрену
- Пайдаланушының орналасқан жерін анықтайтын мобильді қосымша құру дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Геолокация деген не
- GPS, Wi-Fi және мобильді желі арқылы орынды анықтау
- Google Maps API мүмкіндіктері
- Орналасу рұқсаттары (Permissions)
- Пайдаланушының ағымдағы орнын алу
- Картаға маркер (Marker) қою
- Геолокация деректерін қолдану салалары

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Google Cloud Console арқылы Maps API кілтін алу
- Google Maps API-ды жобаға қосу
- AndroidManifest.xml файлына қажетті рұқсаттарды (ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION) енгізу
- Fused Location Provider арқылы GPS деректерін алу
- Картада пайдаланушы орнын көрсету

- Қосымшаны эмуляторда және нақты құрылғыда тексеру
- Геолокацияны қолдану кезінде құпиялық талаптарын сақтау

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Геолокация деректері Model ретінде, карта View ретінде қолданылады
MVP (Model–View–Presenter)	Presenter GPS деректерін өңдеп, View-ге жібереді
MVVM (Model–View–ViewModel)	ViewModel орналасу деректерін бақылап, UI-ды жаңартады
Clean Architecture	Геолокация логикасы UI қабатынан тәуелсіз ұйымдастырылады

Тапсырмалар

- Google Maps API кілтін алып, жобаға қосу
- Қосымшаға карта компонентін енгізу
- Пайдаланушыдан геолокация рұқсатын сұрау
- Ағымдағы орналасқан жерді анықтау
- Картада маркер қою және координаттарды көрсету
- Қосымшаны эмуляторда немесе нақты құрылғыда іске қосып тексеру

Бақылау сұрақтары

- Геолокация дегеніміз не
- GPS қалай жұмыс істейді
- Google Maps API қандай мүмкіндіктер береді
- Геолокация рұқсаттары не үшін қажет
- Пайдаланушы орнын анықтаудың қандай тәсілдері бар
- Геолокацияны қолдануда қандай қауіпсіздік талаптары сақталуы тиіс

Студенттің өздік жұмыстары

- Картада бірнеше маркерден тұратын қосымша жасау
- Пайдаланушы қозғалысын картада көрсету
- Геолокацияны қолданатын танымал қосымшаларға мысал келтіру
- Google Maps API мен басқа карта сервистерін салыстыру
- Геолокация технологиясының артықшылықтары мен кемшіліктері туралы қысқаша эссе жазу

№13. Тәжірибелік сабақ.

Тақырыбы: Фондық процестер және хабарламалар (Notifications, Service, BroadcastReceiver)

Сабақтың мақсаты

- Мобильді қосымшалардағы фондық процестер ұғымын түсіндіру
- Service, BroadcastReceiver компоненттерінің қызметін меңгеру
- Notifications арқылы пайдаланушыға хабарлама жіберу тәсілдерін үйрету
- Қосымшаның фондық режимде жұмыс істеу мүмкіндіктерін тәжірибеде көрсету

Қарастырылатын сұрақтар

- Фондық процесс деген не
- Android жүйесіндегі негізгі компоненттер
- Service түрлері (Foreground, Background)
- BroadcastReceiver ұғымы және қолданылуы
- Notifications қызметі
- Қосымшаның өмірлік циклі және фондық шектеулер
- Энергияны үнемдеу және жүйе талаптары

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Жобаға Service компонентін қосу
- Қарапайым фондық қызметті іске қосу
- BroadcastReceiver арқылы жүйелік оқиғаларды қабылдау
- Notification Channel құру және хабарлама көрсету
- Фондық процестердің жүйе ресурстарына әсерін ескеру
- Қосымшаны эмуляторда немесе нақты құрылғыда тестілеу

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Фондық логика Controller-де, хабарламалар View-де көрсетіледі
MVP (Model–View–Presenter)	Presenter фондық процестерді басқарып, View-ге хабарлама жібереді
MVVM (Model–View–ViewModel)	ViewModel фондық деректерді бақылап, UI-ды жаңартады

Архитектуралық үлгі	Сипаттамасы
Clean Architecture	Фондық қызметтер UI қабатынан толық бөлініп ұйымдастырылады

Тапсырмалар

- Қарапайым Service құру және іске қосу
- Service арқылы фондық есеп (мысалы, таймер) орындау
- BroadcastReceiver көмегімен жүйелік оқиғаны (желінің қосылуы, зарядтау) анықтау
- Notification арқылы пайдаланушыға хабарлама жіберу
- Foreground Service қолдану арқылы тұрақты хабарлама көрсету
- Фондық қызметтің жұмысын тексеру

Бақылау сұрақтары

- Фондық процесс дегеніміз не
- Service қандай жағдайда қолданылады
- BroadcastReceiver не үшін қажет
- Notifications пайдаланушыға қандай ақпарат береді
- Foreground Service пен Background Service айырмашылығы неде
- Фондық процестерге қандай шектеулер қойылады

Студенттің өздік жұмыстары

- Фондық режимде жұмыс істейтін шағын қосымша жасау
- Notifications арқылы жоспарланған хабарлама көрсету
- BroadcastReceiver қолданылатын нақты мысалдарды зерттеу
- Android жүйесіндегі фондық шектеулер туралы конспект жазу
- Фондық процестерді тиімді пайдаланудың маңыздылығы туралы қысқаша эссе дайындау

№14. Тәжірибелік сабақ.

Тақырыбы: Мобильді қосымшадағы желілік байланыс (REST API, Retrofit, JSON)

Сабақтың мақсаты

- Мобильді қосымшалардағы желілік байланыс ұғымын түсіндіру
- REST API арқылы сервермен деректер алмасу принциптерін меңгеру
- Retrofit кітапханасын қолдану арқылы HTTP сұраныстарды орындау
- JSON форматындағы деректерді өңдеу дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Желілік байланыс деген не
- Клиент–сервер архитектурасы
- REST API ұғымы және оның қағидалары
- HTTP әдістері (GET, POST, PUT, DELETE)
- JSON деректер форматы
- Retrofit кітапханасының қызметі
- Асинхронды сұраныстар және қателерді өңдеу

Әдістемелік нұсқаулар

- Жұмысты Android Studio ортасында орындау
- Қосымшаға интернет рұқсатын (INTERNET permission) қосу
- Retrofit және JSON өңдеуге арналған тәуелділіктерді Gradle арқылы енгізу
- API интерфейсін анықтау
- Серверден деректерді алу және оларды модель класқа түрлендіру
- Алынған деректерді экранда көрсету (TextView немесе RecyclerView арқылы)
- Желілік қателерді өңдеу және пайдаланушыға хабарлау
- Қосымшаны эмуляторда немесе нақты құрылғыда тексеру

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	REST API-дан алынған деректер Model ретінде, UI View ретінде қолданылады
MVP (Model–View–Presenter)	Presenter Retrofit арқылы деректерді алып, View-ге ұсынады
MVVM (Model–View–ViewModel)	ViewModel желілік деректер ағынын басқарып, UI-ды жаңартады
Clean Architecture	Желілік деректер қабаты UI-ден толық ажыратылады

Тапсырмалар

- Қарапайым REST API таңдап, оның құрылымын зерттеу
- Retrofit көмегімен серверге GET сұраныс жіберу
- JSON форматындағы деректерді оқу және өңдеу
- Алынған деректерді экранда көрсету
- Желілік қате туындаған жағдайда хабарлама шығару
- REST API арқылы алынған деректерді жергілікті түрде сақтау мүмкіндігін талдау

Бақылау сұрақтары

- Желілік байланыс не үшін қажет
- REST API дегеніміз не
- HTTP әдістерінің айырмашылығы қандай
- JSON форматының артықшылықтары неде
- Retrofit не үшін қолданылады
- Желілік сұраныстармен жұмыс істегенде қандай қауіпсіздік талаптары бар

Студенттің өздік жұмыстары

- Ашық REST API пайдаланып шағын қосымша жасау
- JSON деректер құрылымын схема түрінде көрсету
- Retrofit пен басқа желілік кітапханаларды салыстыру
- Желілік байланыс қолданылатын мобильді қосымшаларға мысалдар келтіру
- Мобильді қосымшалардағы клиент–сервер өзара әрекеттесуі туралы қысқаша эссе жазу

№15. Тәжірибелік сабақ.

Тақырыбы: Қосымшаны тестілеу және оны Play Market немесе App Store-ға дайындау

Сабақтың мақсаты

- Мобильді қосымшаны тестілеудің маңызын түсіндіру
- Қателерді анықтау және түзету әдістерін меңгеру
- Android (Play Market) және iOS (App Store) жариялау талаптарымен танысу
- Қосымшаны релизге дайындау (build, signing, versioning) дағдыларын қалыптастыру

Қарастырылатын сұрақтар

- Тестілеу деген не және не үшін қажет
- Тестілеу түрлері (unit, UI, integration, beta)
- Debug және Release режимдерінің айырмашылығы
- Қосымшаны құрастыру (build) және қолтаңбалау (signing)
- VersionCode/VersionName және релиз нұсқасын басқару
- Play Console және App Store Connect платформалары
- Скриншот, сипаттама, иконка, құпиялық (privacy) талаптары
- Қосымша қауіпсіздігі және қолданушы деректерін қорғау

Әдістемелік нұсқаулар

- Жұмысты Android Studio және/немесе Xcode ортасында орындау

- Тестілеу сценарийлерін алдын ала құрастыру (қолданушы әрекеттері, қате жағдайлар)
- Эмуляторда және нақты құрылғыда тексеру
- Логтарды оқу (Logcat / Xcode Console) арқылы қателерді табу
- Release build жасау алдында ресурстарды оңтайландыру (иконка, суреттер, өлшемдер)
- Android үшін: keystore жасап, APK/AAB файлды қолтаңбалау
- iOS үшін: сертификаттар, provisioning profile, archive және TestFlight
- Маркетке жариялау үшін қажетті материалдарды дайындау (сипаттама, скриншот, категория)

Кесте 1 – Архитектуралық үлгілер

Архитектуралық үлгі	Сипаттамасы
MVC (Model–View–Controller)	Құрылым дұрыс болса, тест жазу және қателерді табу жеңілдейді
MVP (Model–View–Presenter)	UI логикасы Presenter-де болғандықтан unit test жүргізу ыңғайлы
MVVM (Model–View–ViewModel)	ViewModel-ді unit test арқылы тексеру кең таралған тәсіл
Clean Architecture	Тестілеуге ең қолайлы: қабаттар бөлек, тәуелділік аз болады

Тапсырмалар

- Қосымшаның негізгі функционалын тексеретін тест жоспарын құру
- Қосымшаны кемінде 2 құрылғыда (эмулятор/нақты) тестілеу
- Қателерді Logcat немесе Console арқылы анықтау және түзету
- Android үшін релиздік жинақ (AAB немесе APK) жасау және қолтаңбалау
- iOS үшін Archive жасау және TestFlight-қа шығару қадамдарын сипаттау
- Play Market және App Store үшін жариялау бетіне қажет материалдар тізімін дайындау

Бақылау сұрақтары

- Тестілеу не үшін қажет
- Unit test және UI test айырмашылығы қандай
- Debug пен Release режимдерінің айырмашылығы неде
- Android-та APK және AAB айырмашылығы қандай
- Қолтаңбалау (signing) не үшін қажет

- Play Console және App Store Connect қандай қызмет атқарады
- Қосымша жариялауда құпиялық саясаты не үшін маңызды

Студенттің өздік жұмыстары

- Өз қосымшасына арналған тест-кейс құрастыру (кемінде 10)
- Play Market және App Store талаптарын салыстырып кесте жасау
- Қосымшаны релизге дайындау чек-парағын (checklist) әзірлеу
- Қолданушы деректерін қорғау бойынша ұсыныстар жинау
- Тестілеу және жариялау тақырыбы бойынша қысқаша конспект жазу